TECHNICAL AND PLAIN-LANGUAGE COMPANION

Explanation Week 5

Vault-backed secrets, role boundaries, repeatable Compose provisioning, and the complete runtime.

This file explains

- Why API, Worker, and Migrator AppRoles can read only their own Vault paths.
- How KeyPerFile maps rendered secret filenames into .NET configuration without exposing Vault tokens.
- Why the one-shot Migrator owns schema changes while ordinary application hosts do not.
- How a second provisioning run proves convergence without resetting persisted volumes or credentials.

FLOW

Vault policy -> short-lived role access -> rendered secret file -> one authorized process

KEY GUARANTEE

Each runtime role receives only the secrets it needs. Provisioning must converge and prove cross-role reads are denied.

Source baseline: current workspace, 25 August 2026.

1. Vault and full Compose runtime

KeyPerFile at every .NET host boundary

```
// API Program.cs:30-31 and each worker Program.cs
builder.Configuration.AddKeyPerFile("/run/secrets", optional: true);
```

A file named `RabbitMq__Password` maps to the nested configuration key `RabbitMq:Password`. The value never needs to appear in tracked JSON or a command-line argument.

Role/service	Secret boundary	Why
API AppRole	API DB connection + JWT signing key	Admits/queries data and issues/verifies access tokens; no broker credential.
Worker AppRole	Worker DB connection + RabbitMQ user/password	Relay and consumers need SQL and broker, but no JWT secret.
Migrator AppRole	Schema-owner DB connection	Only the one-shot migrator changes schema.

Provisioning sequence

1. Validate Docker, shell tools, Windows/NTFS ACL guard, runtime ownership, and persisted-state consistency.
2. Initialize/unseal persisted Vault or reuse matching protected recovery material.
3. Create policies/AppRoles and render files using short-lived role tokens, not the root token.
4. Assert cross-role reads fail.
5. Start SQL/RabbitMQ, reconcile ordinary app and schema-owner database logins, and run the one-shot migrator.
6. Start API, relay, primary worker, DLQ worker, Next, and optional Angular; wait for health.
7. Run end-to-end smoke. Only after success, establish/compare the Vault-held secret fingerprint and write non-secret evidence.

> **Layman's explanation:** Vault is a cabinet with three drawer badges. The provisioner uses each badge briefly to place sealed keys at the correct desk. A desk receives its key, not a master badge.

> On Windows/NTFS, Unix mode bits are not enough. The provisioner validates a Windows ACL restricted to the current user, SYSTEM, and Administrators. Never publish runtime secrets, AppRole material, Vault init/recovery material, or raw logs containing request bodies.

Why the second provision matters

It proves the process converges without resetting volumes or silently replacing credentials that persisted SQL/RabbitMQ state still expects. Evidence should show `vault_cross_role_reads=denied` and `secret_set_fingerprint_status=stable`.

Review summary

Week 5 boundaries, evidence, and short answers

Security boundary

- The API receives its database connection and JWT signing key, but no RabbitMQ credential.
- Workers receive database and RabbitMQ credentials, but no JWT signing key.
- The Migrator receives the schema-owner database connection and runs as a one-shot process.
- Runtime files contain rendered application secrets, never a Vault root token.

Operational checks

- Cross-role reads must be denied and the Vault-held secret-set fingerprint must remain stable on rerun.
- On Windows/NTFS, validate an ACL restricted to the current user, SYSTEM, and Administrators.
- Never publish runtime secrets, AppRole material, Vault initialization or recovery material, or raw sensitive logs.

Executable evidence

- Week 5 provisioning and smoke evidence prove health, least privilege, and a non-destructive second run.

The detailed pages preserve the original source excerpts; headings and footers are renumbered for this standalone file.